

课程笔记

[LLM Agents SP25] Multimodal Autonomous AI Agents

基于 Ruslan Salakhutdinov 授课内容整理

2026-04-02



The slide has a dark blue background. On the left, there is a circular portrait of Ruslan Salakhutdinov, a man with dark hair wearing a light green polo shirt. Below the portrait, his name 'RUSLAN SALAKHUTDINOV' is written in white capital letters, followed by 'VP OF RESEARCH' in smaller white capital letters, and the Meta logo (an infinity symbol) and the word 'Meta' in white. On the right side, the text 'UC Berkeley Center for Responsible, Decentralized Intelligence' is in small white font. Below it, 'Advanced Large Language Model Agents MOOC' is in large white font. A horizontal white line separates this from 'Multimodal Autonomous AI Agents', which is also in large white font. Another horizontal white line separates this from 'March 10th, 2025', which is in white font. In the bottom right corner, there is a decorative pattern of white dots arranged in a grid.

UC Berkeley Center for Responsible,
Decentralized Intelligence

Advanced Large Language
Model Agents MOOC

Multimodal
Autonomous AI Agents

March 10th, 2025

RUSLAN
SALAKHUTDINOV
VP OF RESEARCH
Meta

视频作者/频道: Berkeley RDI

发布日期: 2025-03-11

视频时长: 1:17:42

视频链接: <https://www.youtube.com/watch?v=RPIN0YM12RU>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 课程定位与核心问题	3
1.1 为什么这讲重要	3
1.2 这门课对实践者的直接启发	3
1.3 本章小结	4
2 VisualWebArena: 面向真实任务的评估基准	4
2.1 从 WebArena 到 VisualWebArena	4
2.2 为什么“人类容易”的任务对 Agent 仍然难	4
2.3 本章小结	5
3 环境形式化: POMDP、状态与动作空间	5
3.1 为什么要做形式化	5
3.2 动作空间设计决定了系统上限	5
3.3 奖励稀疏下的训练与评估挑战	6
3.4 本章小结	7
4 推理时搜索: 从单轨执行到可回溯决策	7
4.1 为什么必须引入搜索	7
4.2 基础策略: Rejection Sampling	7
4.3 进阶策略: 价值引导与树搜索	7
4.4 本章小结	8
5 规划模型与执行模型解耦	8
5.1 为什么需要 Planner-Executor 分工	8
5.2 层级接口如何设计	9
5.3 何时该重新规划	9
5.4 本章小结	10
6 Verifier 与 LLM-as-a-Judge: 把评估嵌入推理	10
6.1 从终局评估到中间评估	10
6.2 LLM-as-a-Judge 的现实价值	10
6.3 Judge 可靠性与校准	10
6.4 本章小结	11
7 数据扩展: 从稀缺轨迹到可持续训练闭环	11
7.1 为什么数据是瓶颈	11
7.2 在线收集与过滤	12
7.3 训练阶段的组合策略	12
7.4 本章小结	12
8 结果解读: 性能爬升、模型差距与计算代价	12
8.1 从低基线到接近人类水平	12
8.2 模型代际演进	13

8.3	成本与收益权衡	14
8.4	本章小结	14
9	失败模式与工程防线	14
9.1	典型失败：误动作后无法恢复	14
9.2	失败模式分类	15
9.3	人类协同仍然必要	15
9.4	本章小结	15
10	研究前沿：从网页 Agent 到通用多模态执行体	16
10.1	更高层的动作抽象	16
10.2	并行搜索与多实例执行	16
10.3	评估基准的下一步	16
10.4	本章小结	16
11	案例拆解：从任务描述到可执行轨迹	16
11.1	以“预订符合约束的餐厅”为例	16
11.2	轨迹级质量控制	17
11.3	可复用的执行模板	18
11.4	本章小结	18
12	工程清单：把研究原型变成生产系统	18
12.1	上线前必须回答的十个问题	18
12.2	最容易被低估的三类工程成本	19
12.3	从指标驱动到价值驱动	19
12.4	本章小结	20
13	总结与延伸	20
13.1	核心结论	20
13.2	总结表	20
13.3	本章小结	20
13.4	Further Reading	20

1 课程定位与核心问题

1.1 为什么这讲重要

这节课并不是再介绍一个“更大模型”，而是把注意力放到一个更工程化的问题：当模型进入真实网页环境，如何从“会回答问题”变成“会完成任务”。Ruslan 把主题压缩成三件事：评估、搜索、数据扩展。这三件事恰好是大多数 Agent 系统落地时最先撞墙的地方。

本讲主线

1. **VisualWebArena**: 如何在可复现环境里评估多模态 Web Agent;
2. **Inference-Time Search**: 如何在推理时让 Agent 更会试错和回溯;
3. **Data Scaling**: 如何在数据稀缺条件下持续提升 Agent 能力。

从“聊天”到“执行”的范式变化

当任务从问答变成网页操作时，模型要同时处理视觉输入、状态变化、动作约束、长期目标和失败恢复。这个问题更接近“Sequential Decision Making”，而不是单轮文本生成。

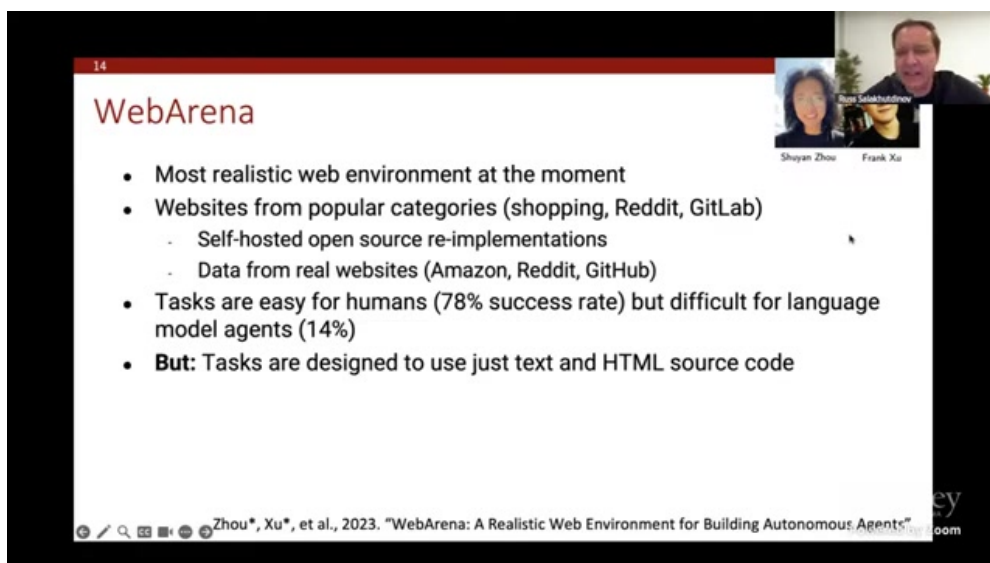


图 1: 视频约 00:06:50: 讲者引入 VisualWebArena，强调在真实交互环境评估多模态 Agent

1.2 这门课对实践者的直接启发

讲者反复强调一点：Agent 系统的瓶颈不只在模型参数，而在系统化能力。即便基础模型更强，如果没有搜索、验证器和高质量交互数据，性能上限仍然会很快触顶。反过来，哪怕基础模型不是最强，只要在这三条链路上做对，也可能拿到可观收益。

常见误区

把“模型能力提升”直接等同于“任务完成率提升”往往是错的。Web 任务里，状态空间大、步骤长、中间错误代价高，单步能力提升并不会自动转化为长链路成功率。

1.3 本章小结

本讲聚焦 Agent 工程的三根支柱：评估、搜索、数据。它关注的不是单次回答质量，而是长链路任务的成功率与可扩展性。

2 VisualWebArena：面向真实任务的评估基准

2.1 从 WebArena 到 VisualWebArena

WebArena 已经把网页交互任务结构化，但纯文本表示有明显局限：很多视觉语义（布局、图片、图表、按钮位置）无法完整保留。VisualWebArena 则把视觉信息显式纳入，要求 Agent 同时理解页面截图和结构化网页元素。

VisualWebArena 解决了什么

- 任务不再只依赖 HTML 文本，而是依赖真实视觉上下文；
- 评估不再只测“能否写出文本答案”，而测“能否完成一段交互轨迹”；
- 错误来源更接近真实系统，包括误点、误读、漏检和回溯失败。

任务类型通常包含三类

1. **视觉检索型**：根据图片或版式定位目标；
2. **跨站流程型**：多标签页、多网站串联完成任务；
3. **条件满足型**：满足价格、评分、时间等约束并完成提交。

2.2 为什么“人类容易”的任务对 Agent 仍然难

讲者提到很多任务对人类并不复杂，但对 Agent 依然困难，核心原因是“决策链条过长且错误可累积”。一个看似小失误（点错按钮、读错页面元素）就会把后续轨迹全部带偏。

评估指标的陷阱

如果只看终局成功率，容易忽略中间失败模式；如果只看中间步骤，又可能高估最终可交付能力。更稳妥的方法是“终局指标 + 轨迹质量 + 错误类型”联合评估。

评估维度	单轮 LLM 常见做法	VisualWebArena 风格做法
输入形式	纯文本 prompt	页面截图 + 结构化网页状态
输出形式	文本答案	多步动作序列
正确性判定	文本匹配	任务终局 + 轨迹一致性
主要失败形态	幻觉/遗漏	误操作/回溯失败/策略偏航

表 1: 从文本问答评测到交互任务评测的范式变化

2.3 本章小结

VisualWebArena 的价值在于把评估目标从“会说”变为“会做”，并把多模态感知、长链路决策和执行稳定性纳入统一框架。

3 环境形式化：POMDP、状态与动作空间

3.1 为什么要做形式化

讲者把 Web Agent 建模为 POMDP，这是非常关键的一步。它让我们可以用强化学习和搜索的统一语言描述问题，而不是把系统行为当作一堆启发式脚本。

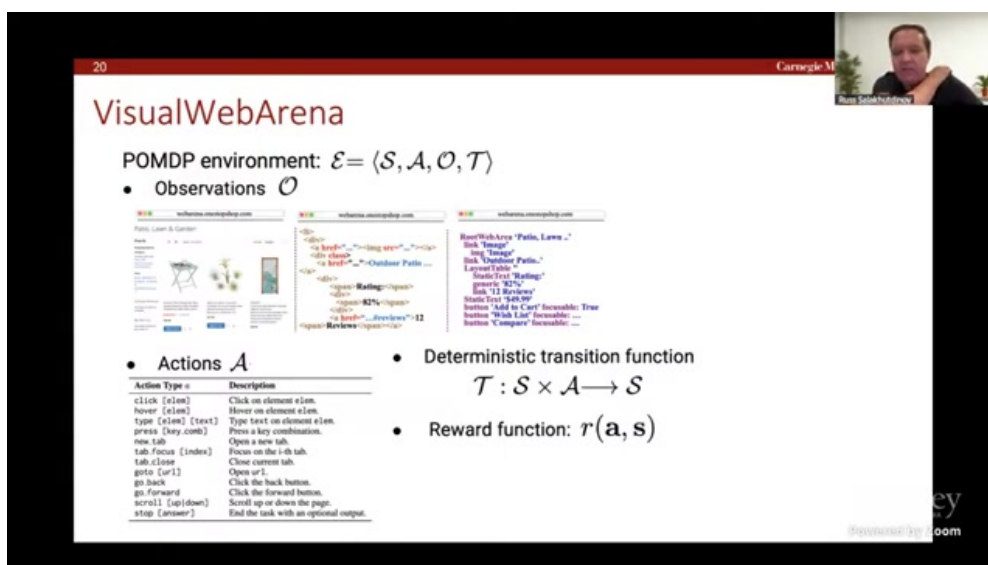


图 2: 视频约 00:13:22: 讲者将网页交互问题写成 states / actions / observations / transitions

POMDP 在这里的直观解释

- **State:** 网页真实状态（包括不可见信息）；
- **Observation:** Agent 当前可见内容（截图、DOM、可访问树等）；
- **Action:** 点击、输入、滚动、切换标签、停止等；
- **Reward:** 任务是否被正确完成，通常较稀疏。

3.2 动作空间设计决定了系统上限

动作空间过粗会导致表达力不足，过细又会导致搜索空间爆炸。实践中通常需要“语义动作 + 低层动作”的分层设计，例如“搜索餐厅”这类子目标由 planner 给出，再由 executor 拆成点击与输入。

动作空间设计原则

1. 原子动作要可执行且可验证；
2. 高层动作要能减少搜索深度；
3. 停止动作必须配套可靠的完成判定；
4. 每步动作都应带可追踪日志，便于回放和调试。

动作层级	典型动作	主要风险
低层动作	click / type / scroll / hover	步数长、局部错误易累积
中层动作	open-site / query / filter	语义歧义、参数绑定困难
高层动作	complete-subtask / stop	过早停止或误判完成

表 2: Web Agent 动作层级与风险

3.3 奖励稀疏下的训练与评估挑战

在网页任务里，很多 reward 只在末尾给出，因此信用分配非常困难。系统若只靠终局信号，很难学会中间步骤的精细纠错能力。

稀疏奖励的副作用

- 训练不稳定，策略更新方差大；
- 模型倾向学会投机 shortcut；
- 长链路任务的中间错误难以被及时惩罚。

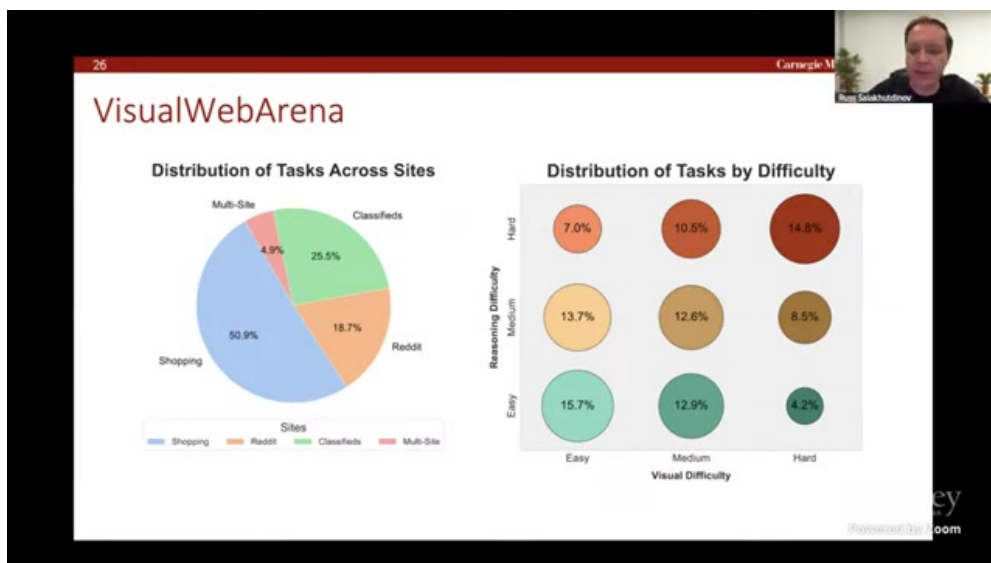


图 3: 视频约 00:16:03: 讲者展示复杂任务轨迹，强调长链路推理与执行一致性

3.4 本章小结

POMDP 形式化让 Agent 任务从“脚本工程”走向“决策问题”。但奖励稀疏与动作空间爆炸使得仅靠单次前向推理难以稳定完成复杂任务。

4 推理时搜索：从单轨执行到可回溯决策

4.1 为什么必须引入搜索

讲者给出的结论非常明确：没有搜索能力，Agent 一旦走偏就很难回到正确轨道。尤其在 Web 场景中，错误动作会改变页面状态，后续操作空间随之变化，恢复成本极高。

搜索的直接收益

- 允许多个候选轨迹并行评估；
- 支持回溯与分支重试；
- 把“一次猜对”变成“多次试探后选优”。

4.2 基础策略：Rejection Sampling

最简单的做法是先生成多条轨迹，再用终局标准筛选。它实现简单、并行友好，但效率不高，因为大量无效轨迹会浪费推理预算。

Rejection Sampling 何时有效

当任务较短、验证器可靠、并行资源充足时，这种方法可以作为低工程成本基线，并快速带来增益。

4.3 进阶策略：价值引导与树搜索

当任务长度增加，纯采样很快失效。此时要引入价值函数或 Judge 估计中间状态潜力，优先扩展高价值节点，实现“边探索边裁枝”。

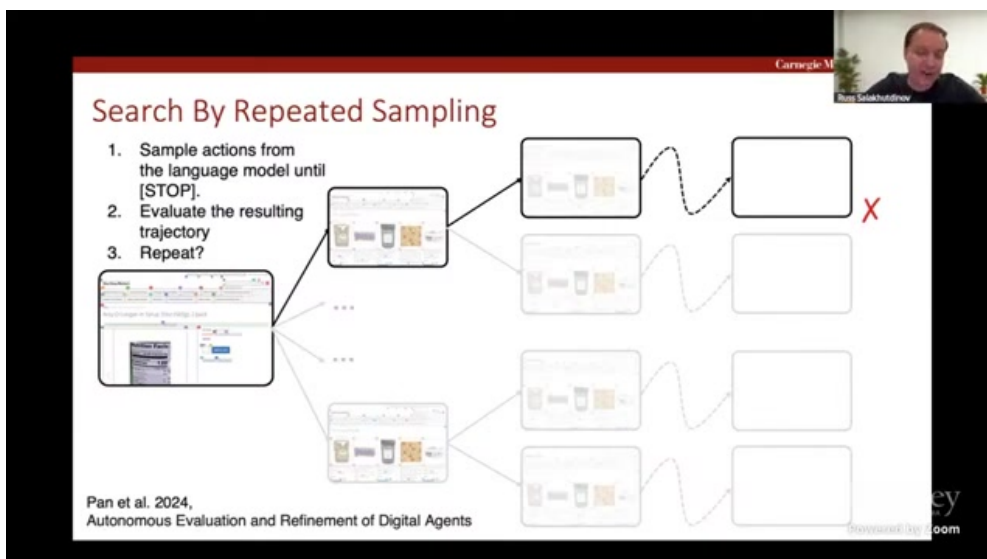


图 4: 视频约 00:34:39: 讲者说明在搜索分支中评估轨迹结果并做继续/终止决策

方法	优点	不足
单轨执行	延迟低、实现简单	错误不可恢复，成功率受单次决策噪声影响大
Rejection Sampling	并行友好，容易上线	计算浪费明显，长任务收益下降
价值引导搜索	样本效率高，可回溯	工程复杂，对 Judge 质量敏感

表 3: Agent 推理策略对比

搜索不是免费午餐

搜索会显著增加推理成本。若没有良好剪枝和节点复用机制，系统可能陷入“预算耗尽但未收敛”的状态。

4.4 本章小结

在长链路网页任务中，搜索能力不是可选项，而是鲁棒性的核心来源。关键不在于“是否搜索”，而在于“如何高效搜索”。

5 规划模型与执行模型解耦

5.1 为什么需要 Planner-Executor 分工

讲者后半段强调了层级化思路：高层模型负责计划，低层模型负责执行。这样可以把复杂任务拆成局部可控子任务，降低一步步直接决策的噪声。

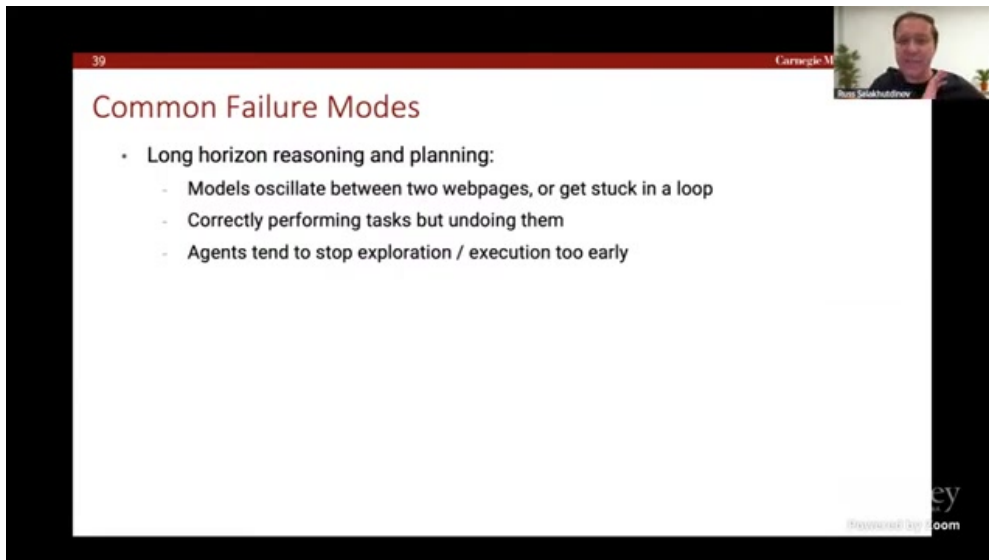


图 5: 视频约 00:27:40: 讲者讨论让 planning model 先产出计划，再由执行模型落地动作

解耦后的收益

- 计划层可做全局目标约束；
- 执行层可针对当前页面做局部最优动作；
- 错误定位更清晰，便于调试和回滚。

5.2 层级接口如何设计

一个可落地接口通常包含三部分：子目标描述、执行边界、完成判定。没有清晰接口时，规划层与执行层会相互覆盖，造成“计划写得好但执行跑偏”的现象。

层级接口最小字段

1. **subgoal**: 当前阶段目标；
2. **allowed actions**: 允许动作集合；
3. **stop condition**: 完成或失败的判定标准；
4. **fallback rule**: 异常时返回上层的条件。

5.3 何时该重新规划

执行中一旦触发高置信度异常（元素不存在、页面跳转异常、预算超限），系统应及时回到规划层，而不是盲目继续下一个动作。

规划层常见失败

- 计划粒度过粗，执行层无法落地；
- 计划粒度过细，失去层级价值；
- 缺少“何时重规划”规则，导致错误持续扩散。

5.4 本章小结

Planner-Executor 解耦是长链路 Agent 的关键工程模式。真正决定效果的是接口设计与重规划策略，而不仅是换一个更大模型。

6 Verifier 与 LLM-as-a-Judge：把评估嵌入推理

6.1 从终局评估到中间评估

如果只在轨迹末尾判定成功，反馈太晚。讲者提出把 verifier 前移到中间步骤，让系统在推理时就能判断“当前路径是否值得继续”。

Verifier 的三类作用

1. 路径打分：为搜索裁枝提供依据；
2. 错误检测：识别明显偏航状态；
3. 数据过滤：筛选可用于训练的高质量轨迹。

6.2 LLM-as-a-Judge 的现实价值

让 LLM 评估轨迹通常比让它直接生成最优轨迹更稳定。因为“判断好坏”通常比“一步到位做对”更容易，这也是本讲反复强调的工程经验。

Judge 提示词的关键要素

- 任务目标与约束；
- 完整轨迹（或关键步骤摘要）；
- 明确输出模板：通过/失败 + 依据；
- 置信度估计，供搜索策略使用。

6.3 Judge 可靠性与校准

Judge 不是绝对真值。若评估标准模糊或提示词漂移，Judge 可能给出高置信度错误结论，反而误导搜索。

Judge 的三种风险

- 奖励黑客：模型学会迎合 Judge 文风而非真实完成任务；
- 领域偏差：Judge 在陌生网页模板下失真；
- 置信度失配：高分不代表高成功率。

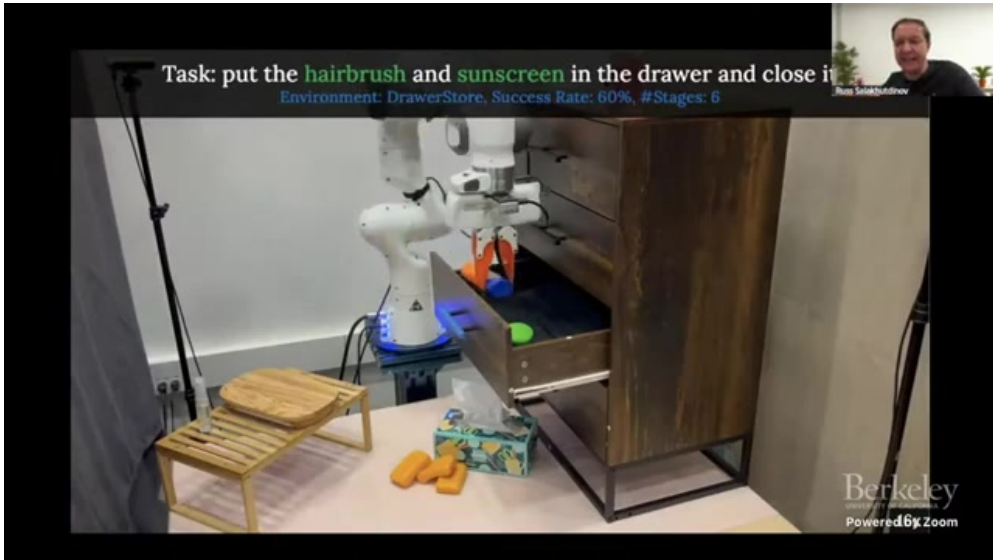


图 6: 视频约 01:12:10: 规划、执行、验证器构成完整闭环，评估不再只发生在任务末尾

6.4 本章小结

Verifier 让 Agent 从“先做再判”升级到“边做边判”。但 Judge 可靠性需要持续校准，否则会把系统引向错误优化方向。

7 数据扩展：从稀缺轨迹到可持续训练闭环

7.1 为什么数据是瓶颈

Web Agent 的高质量交互数据比纯文本语料更昂贵。每条轨迹都包含环境状态、动作序列、工具结果和终局判定，采集与清洗成本都高。

数据难点

- 真实交互慢，单位样本成本高；
- 轨迹长，标注和验证复杂；
- 环境随时间变化，旧数据会过时。

7.2 在线收集与过滤

讲者建议在真实或高保真模拟环境中在线采样，再通过 verifier/Judge 做分层过滤：先去除明显失败，再保留高置信度成功轨迹，最后按任务类型均衡采样。

可执行的数据流水线

1. 采样：多策略并行生成轨迹；
2. 过滤：终局判定 + 过程判定；
3. 重加权：困难样本提升权重；
4. 训练：SFT 或 RL 更新；
5. 回放：在线 A/B 检验是否真提升。

7.3 训练阶段的组合策略

实践中常见组合是“SFT 打底 + 搜索增强 + RL 微调”。SFT 提供稳定起点，搜索提供高质量探索轨迹，RL 在交互反馈下做策略细化。

阶段	目标	注意事项
SFT	学会基本动作语法与任务格式	防止只记模板，忽视长链路泛化
Search-augmented	提升探索质量与成功轨迹占比	控制推理预算，避免过度采样
RL / policy update	对齐终局成功率和鲁棒性	防止 reward hacking 与分布偏移

表 4: Web Agent 常见训练组合路径

数据扩展中的隐性风险

如果只追求成功率，系统可能偏向“容易任务”，导致困难场景性能退化。数据扩展必须和任务难度分布管理同时进行。

7.4 本章小结

Agent 能力扩展的核心不只是模型升级，而是可持续数据闭环。在线采样、可靠过滤和难度均衡决定了系统是否能稳定进步。

8 结果解读：性能爬升、模型差距与计算代价

8.1 从低基线到接近人类水平

讲者展示了通过更好的推理策略和评估机制，系统性能可以从较低水平显著提升，并在部分设置下接近人类表现。这一结果说明“系统设计”对性能提升同样关键。

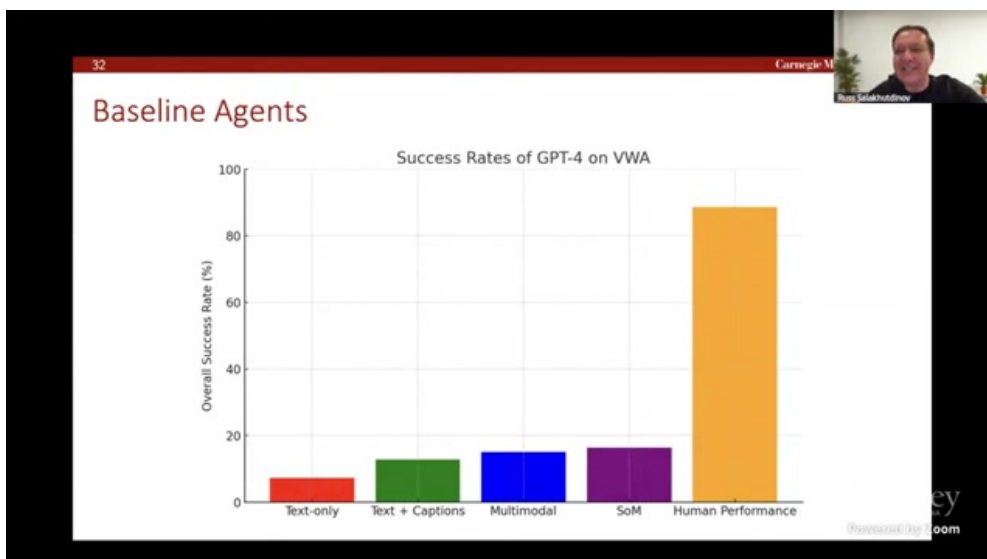


图 7: 视频约 00:22:23: 讲者展示性能提升并提到接近 human level 的区间

这组结果的真正含义

它并不代表 Agent “已经解决 Web 自动化”，而是说明在正确的评估和搜索框架下，长链路任务存在明确可优化空间。

8.2 模型代际演进

讲者提到 Gemini Pro、GPT-4 以及后续模型在该类任务上的爬升趋势。模型能力的确在进步，但系统中“计划-执行-验证”的配套仍是必要条件。

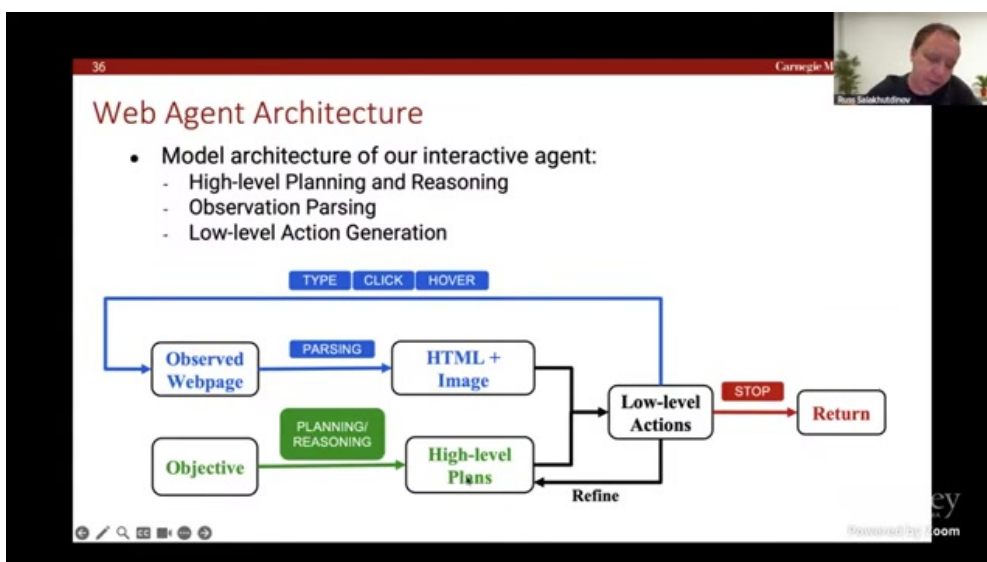


图 8: 视频约 00:25:15: 讲者讨论 Gemini Pro、GPT-4 与后续模型在任务表现上的提升

如何看待模型提升

- 强模型降低了单步错误率；
- 但长链路稳定性仍受搜索和回溯机制影响；
- 模型越强，越要配套更严格的评估与治理，避免虚高指标。

8.3 成本与收益权衡

搜索和验证带来成功率提升，同时也增加 token 与环境调用成本。生产系统必须显式管理“成功率收益 / 成本开销”的平衡。

成本管理常见失误

- 只追求成功率，忽略单位任务成本；
- 忽略失败重试的尾部开销；
- 没有任务分级策略，导致简单任务也走重搜索路径。

8.4 本章小结

实验结果表明 Agent 性能可通过系统化方法显著爬升。但从研究到上线仍需同时优化成功率、时延和成本，而不是单点追分。

9 失败模式与工程防线

9.1 典型失败：误动作后无法恢复

讲者在问答环节举了很典型的例子：模型执行了错误动作后，若缺少搜索和回溯，后续很难自我纠正，最终完全偏离任务目标。

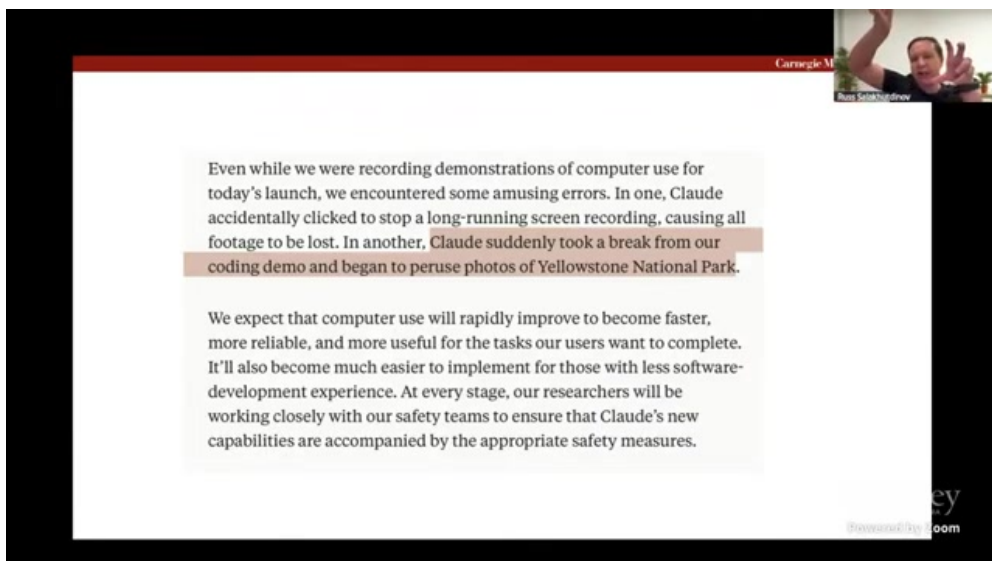


图 9: 视频约 01:17:20: 讲者讨论错误动作后的恢复困难，强调搜索与回溯机制必要性

故障恢复的最小闭环

1. 检测：快速识别当前轨迹偏航；
2. 回滚：回到最近可靠状态；
3. 重规划：更新子目标与动作约束；
4. 复核：通过 verifier 判定是否重回正轨。

9.2 失败模式分类

失败类型	典型症状	工程修复手段
感知错误	读错按钮/表单字段/图像内容	多模态校验、元素置信度阈值
规划错误	子目标顺序错误或约束遗漏	计划模板、子目标检查器
执行错误	连续误点、误输入、误停止	回放日志、状态快照与回滚
评估错误	Judge 误判成功/失败	多 Judge 交叉、人工抽检校准

表 5: Web Agent 常见失败模式与修复策略

不要把“失败”当成噪声样本

失败轨迹是最有价值的数据来源之一。系统应系统化归档失败类型，把它们喂回训练和评估流程，形成持续改进回路。

9.3 人类协同仍然必要

对于高风险任务，Agent 应具备“请求澄清”和“升级到人工”的能力。人类不是最后兜底才出现，而应在不确定性升高时及时介入。

人机协同触发条件示例

- 连续两次高置信度分歧（planner 与 verifier 不一致）；
- 关键字段不可判定（支付、身份、隐私相关）；
- 预算超限但任务未收敛。

9.4 本章小结

Agent 失败模式可被系统化管理。真正可靠的系统不是“永不出错”，而是“出错可检测、可恢复、可学习”。

10 研究前沿：从网页 Agent 到通用多模态执行体

10.1 更高层的动作抽象

讲者提出一个值得长期投入的方向：为 Agent 设计更高层语义抽象，让模型不必在每个任务都从 click/type 原子动作开始搜索。这能显著降低搜索深度和工程复杂度。

高层抽象的潜在收益

- 减少低层动作噪声；
- 提高跨站点泛化；
- 为多 Agent 协同创造统一接口。

10.2 并行搜索与多实例执行

讲者还提到并行化的重要性：同一任务可在多个实例上探索不同路径，再把最优轨迹回传主流程。这与大模型时代“test-time compute”增长趋势一致。

并行执行的设计要点

1. 子任务可独立回放；
2. 共享中间评估信号；
3. 统一冲突解决策略（例如取最高价值路径）。

10.3 评估基准的下一步

下一代基准需要更真实地覆盖：动态网站、个性化页面、长期会话、跨设备交互和多角色协作。否则系统会在静态 benchmark 上提升，却在真实生产环境失效。

Benchmark 与真实部署之间的结构性差距

网页产品持续变化、元素动态加载、权限和会话约束复杂，任何固定 benchmark 都可能快速过时。评估体系必须支持持续更新与版本追踪。

10.4 本章小结

多模态 Agent 的长期突破点在于动作抽象、并行搜索和动态评估体系。未来竞争将更多发生在系统工程层，而不仅是参数规模层。

11 案例拆解：从任务描述到可执行轨迹

11.1 以“预订符合约束的餐厅”为例

讲座中多次以餐厅检索类任务说明 Agent 的执行难点。这类任务看似简单，但在真实网页里通常包含多重约束：地点、价格、评分、营业时间、菜系偏好以及下单流程。系统如果没有显式状态管理，很容易在中途忘记条件。

任务拆解的推荐结构

1. **约束抽取**：把自然语言需求转为结构化字段；
2. **候选检索**：在站内或跨站生成候选集合；
3. **证据核验**：逐项核验硬约束（如价格上限）；
4. **动作执行**：进入下单流程并完成关键字段填写；
5. **终局确认**：通过页面状态与 verifier 双重确认任务完成。

为什么“结构化约束”很关键

如果不把约束结构化，Agent 往往会在长链路中丢失关键信息，只保留模糊偏好。把约束显式存成键值对后，可在每一步执行前做一致性检查，显著降低“后期才发现不满足条件”的返工成本。

11.2 轨迹级质量控制

在该任务中，单步正确并不代表轨迹正确。例如模型可能先选到看起来不错的餐厅，但价格或可用时间不满足要求。轨迹级质量控制要检查“目标一致性”，而不是只检查“动作可执行性”。

检查层	检查对象	常见错误
字段一致性	价格、时间、位置、评分	字段遗漏、单位混淆、约束冲突
页面一致性	当前页面是否包含目标证据	元素识别错误、读错卡片信息
流程一致性	是否进入正确下单路径	错站点、错入口、提前停止
终局一致性	任务结果是否与原需求一致	局部成功但全局不满足

表 6: 餐厅检索任务的轨迹级质量控制

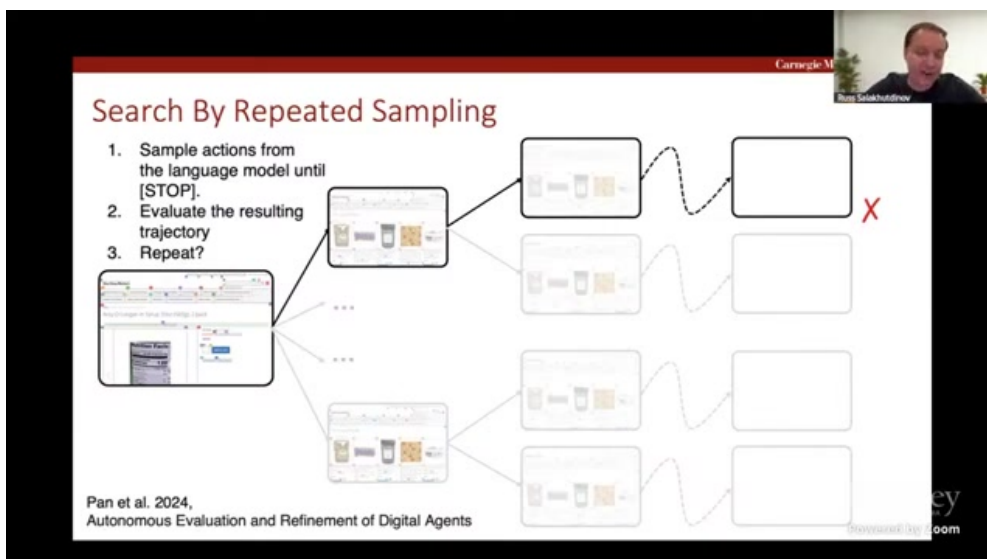


图 10: 视频约 00:34:39: 在候选轨迹间比较后再继续扩展，体现“先评估后执行”的策略

11.3 可复用的执行模板

为了减少每次任务都从零探索，可维护“任务模板库”。模板不是固定脚本，而是“可约束的流程骨架”：定义阶段顺序、关键检查点与回滚策略。这样既保留灵活性，又能显著降低长任务方差。

模板化的边界

模板过刚会损害泛化，模板过松又无法约束错误。合理做法是将模板用于高风险步骤（支付、身份、确认页），而将探索留给低风险步骤（候选检索与排序）。

11.4 本章小结

复杂网页任务的关键不在于“让模型更会猜”，而在于“让轨迹更可控”。结构化约束、轨迹级检查和可复用模板是把演示系统变成稳定系统的三件基础工作。

12 工程清单：把研究原型变成生产系统

12.1 上线前必须回答的十个问题

在研究原型阶段，系统往往追求尽快看到成功案例；但走向生产时，必须把鲁棒性、成本、可审计性前置。下面这张清单可作为上线前评审模板。

#	问题	通过标准示例
1	任务边界是否清晰?	输入约束、禁止动作、终止条件均有文档化定义
2	状态是否可回放?	每步 observation/action/result 均可重放与审计
3	搜索预算是否可控?	设置最大步数、最大分支、超时回退策略
4	Judge 是否校准?	离线对照集上精度达标, 且有定期复核机制
5	失败能否自动恢复?	支持异常检测、回滚、重规划三段式恢复
6	人工介入触发是否明确?	对支付、隐私、低置信度场景有强制升级策略
7	数据回流是否闭环?	成功/失败轨迹都能回流并参与下一轮训练
8	成本是否可解释?	每类任务有 token、时延、外部调用成本统计
9	评估是否持续更新?	benchmark 与真实流量都做持续监控
10	安全与合规是否覆盖?	敏感操作有最小权限和审计日志

表 7: Web Agent 生产化前的最小工程清单

12.2 最容易被低估的三类工程成本

1. **状态管理成本**: 无状态原型很快, 带回滚与多分支状态后复杂度陡增;
2. **评估成本**: 构建高质量对照集、持续校准 Judge 比训练一个 demo 更费时;
3. **运维成本**: 网页结构变化频繁, 解析规则和动作映射需要持续维护。

为何“运维”会成为主成本

很多团队在 PoC 阶段只看模型效果, 忽视网页变化导致的规则失效。上线后真正消耗人力的, 常常是 selector 变更、页面流程改版、反自动化机制升级等持续运维问题。

12.3 从指标驱动到价值驱动

讲者谈到 “economically valuable tasks”, 这提醒我们: 不要只看 benchmark 分数, 而要把任务价值纳入优化目标。一个高分但低价值任务, 不应占用大量推理预算; 反之, 高价值任务应允许更高搜索成本和更强人工协同。

价值分层调度建议

- 低价值任务: 轻量策略 + 低预算 + 快速失败;
- 中价值任务: 中等搜索 + 自动回滚;
- 高价值任务: 强搜索 + 多重验证 + 人工介入。

12.4 本章小结

Agent 生产化的核心难点不在“能否跑通 demo”，而在“能否稳定、可控、可审计地持续运行”。工程清单和价值分层调度是跨过这道门槛的关键抓手。

13 总结与延伸

13.1 核心结论

本讲可执行结论

1. 先建立真实交互评估，再谈模型优化；
2. 推理时搜索是长链路任务鲁棒性的核心；
3. Verifier/Judge 是连接搜索、训练和数据过滤的关键组件；
4. 数据闭环决定能力能否持续增长；
5. 失败恢复机制与人机协同必须在系统设计早期纳入。

13.2 总结表

主题	讲者重点	工程动作
评估	VisualWebArena 衡量真实交互能力	建立终局 + 轨迹 + 错误类型联合指标
搜索	单轨执行不稳，需可回溯决策	上线分层搜索与预算控制
验证器	Judge 更擅长“评”而非“做”	把 verifier 前移到推理中间步骤
数据扩展	在线采样 + 过滤形成闭环	构建失败样本回流与难度重加权
鲁棒性	失败恢复与人工协同不可省略	增加回滚、重规划、澄清触发机制

表 8: Multimodal Autonomous AI Agents 讲座的工程落地摘要

13.3 本章小结

这节课给出的不是单一算法，而是一套 Agent 工程方法论：以真实评估为锚点，用搜索和验证器提升推理鲁棒性，用数据闭环维持长期迭代速度。

13.4 Further Reading

- VisualWebArena / WebArena 相关论文与基准代码；
- LLM-as-a-Judge 与 trajectory-level verifier 研究；
- Search-Augmented Inference 在 Agent 任务上的方法比较；

- 在线交互数据收集与高置信度轨迹过滤实践；
- 多智能体规划与并行执行框架（用于高复杂度长任务）。